

# Namespace ExBrainSdk

## Classes

### [ExBrainApi](#)

ExBrain low-level API クラス / ExBrain low-level API class

## Delegates

### [ExBrainApi.NotifyMeasureDatasCallBack](#)

装置計測データ通知用コールバック関数 / Callback for device measurement data notification

### [ExBrainApi.NotifyProcessMessageCallBack](#)

テキストメッセージ通知用コールバック関数 / Callback for text message notification

### [ExBrainApi.NotifySearchDevicesCallBack](#)

装置検索結果通知用コールバック関数 / Callback for device search result notification

### [ExBrainApi.NotifyStatusChangedCallBack](#)

状態変化通知用コールバック関数 / Callback for status change notification

# Class ExBrainApi

Namespace: [ExBrainSdk](#)

Assembly: ExBrainSdk.dll

ExBrain low-level API クラス / ExBrain low-level API class

```
public class ExBrainApi
```

## Inheritance

[object](#) ← ExBrainApi

## Inherited Members

[object.Equals\(object\)](#), [object.Equals\(object, object\)](#), [object.GetHashCode\(\)](#), [object.GetType\(\)](#), [object.MemberwiseClone\(\)](#), [object.ReferenceEquals\(object, object\)](#), [object.ToString\(\)](#)

## Constructors

ExBrainApi(NotifyStatusChangedCallback?,  
NotifyProcessMessageCallback?, bool)

ExBrainApi インスタンスを生成する / Creates an ExBrainApi instance

```
public ExBrainApi(ExBrainApi.NotifyStatusChangedCallback? notifyStatusChangedCallback =  
null, ExBrainApi.NotifyProcessMessageCallback? notifyProcessMessageCallback = null, bool  
waitReconnectionWhenLost = false)
```

## Parameters

**notifyStatusChangedCallback** [ExBrainApi.NotifyStatusChangedCallback](#)

状態変化通知用コールバック / Callback for status change notification

**notifyProcessMessageCallback** [ExBrainApi.NotifyProcessMessageCallback](#)

テキストメッセージ通知用コールバック / Callback for text message notification

**waitReconnectionWhenLost** [bool](#)

切断時再接続フラグ / Reconnect on disconnect flag

## Properties

### Calibration

現状のキャリブレーション情報の参照 / Current calibration information reference

```
public ExBrainCalibration? Calibration { get; }
```

Property Value

[ExBrainCalibration](#)

### ConnectType

接続方式 / Connection type

```
public EnmSdkConnectType ConnectType { get; }
```

Property Value

[EnmSdkConnectType](#)

### CurrentStatus

SDK動作状態 / SDK operation status

```
public EnmSdkStatus CurrentStatus { get; }
```

Property Value

[EnmSdkStatus](#)

### DiscoveredDevice

装置検索結果リスト / Device search result list

```
public ExBrainDiscoveredDevice DiscoveredDevice { get; }
```

Property Value

[ExBrainDiscoveredDevice](#)

## Gain

現在ゲイン値参照 / Current gain value reference

```
public ExBrainGainInfo? Gain { get; }
```

Property Value

[ExBrainGainInfo](#)

## HandleId

装置接続ID / Device connection ID

```
public string HandleId { get; }
```

Property Value

[string](#)

## HwInfo

ハードウェア情報データ / Hardware information data

```
public DeviceHwInfo? HwInfo { get; }
```

Property Value

[DeviceHwInfo](#)

## HwState

ハードウェア状態データ / Hardware state data

```
public DeviceHwState? HwState { get; }
```

Property Value

[DeviceHwState](#)

## IsConnected

装置接続済み状態 / Device connected state

```
public bool IsConnected { get; }
```

Property Value

[bool](#)

## IsInitiated

SDK初期化済み状態 / SDK initialized state

```
public bool IsInitiated { get; }
```

Property Value

[bool](#)

## LogFolder

ログ出力先フォルダ / Log output folder

```
public string? LogFolder { get; }
```

Property Value

[string](#) 

## ProductName

SDK製品名称 / SDK product name

```
public static string ProductName { get; }
```

Property Value

[string](#) 

## ProductVersion

SDK製品バージョン / SDK product version

```
public static string ProductVersion { get; }
```

Property Value

[string](#) 

## SettingsFolder

設定フォルダ / Settings folder

```
public string? SettingsFolder { get; }
```

Property Value

[string](#) 

# Tag

タグ / Tag

```
public string? Tag { get; set; }
```

Property Value

[string](#)

## Methods

### ConnectDeviceAsync(string, EnmSdkConnectType)

装置接続 / Connect device

```
public Task<EnmSdkResult> ConnectDeviceAsync(string handleId, EnmSdkConnectType connectType)
```

Parameters

**handleId** [string](#)

装置接続ID / Device ID for connection

**connectType** [EnmSdkConnectType](#)

接続方式 / Connection type

Returns

[Task](#) <[EnmSdkResult](#)>

結果コード / Result code

### DisconnectDeviceAsync()

装置切断 / Disconnect device

```
public Task DisconnectDeviceAsync()
```

Returns

[Task](#)

## ExitApi()

API終了 / API termination

```
public void ExitApi()
```

## GetBatteryStateAsync(string)

バッテリー状態取得 / Get battery state

```
public Task<BatteryState?> GetBatteryStateAsync(string handleId)
```

Parameters

**handleId** [string](#)

装置接続ID / Device connection ID

Returns

[Task](#) <[BatteryState](#)>

バッテリー状態データ / Battery state data

## InitApi(bool, bool, string?, string?, string?)

API初期化 / API initialization

```
[SuppressMessage("CodeQuality", "IDE0060:Remove unused parameter", Justification =  
"needOutputCommLog, settingsDir, and tag is not implemented yet")]
```



```
public EnmSdkResult InitApi(bool needOutputTraceLog = false, bool needOutputCommLog = false,  
string? logDir = null, string? settingsDir = null, string? tag = null)
```

## Parameters

needOutputTraceLog [bool](#)

トレースログ出力有無 / Output trace log flag

needOutputCommLog [bool](#)

通信ログ出力有無 / Output communication log flag

logDir [string](#)

ログフォルダパス / Log folder path

settingsDir [string](#)

設定フォルダパス / Settings folder path

tag [string](#)

タグ / Tag

## Returns

[EnmSdkResult](#)

結果コード / Result code

## Remarks

[1.1.5] modify 引数にtag追加 / [1.1.5] added 'tag' argument

## PerformCalibrationAsync(string)

キャリブレーション開始 / Start calibration

```
public Task<EnmSdkResult> PerformCalibrationAsync(string handleId)
```

## Parameters

**handleId** [string](#)

装置接続ID / Device connection ID

Returns

[Task](#) <[EnmSdkResult](#)>

結果コード / Result code

## PerformGainAdjustmentAsync(string, string)

ゲイン調整開始 / Start gain adjustment

```
public Task<EnmSdkResult> PerformGainAdjustmentAsync(string handleId, string
mgcParamLogFolder = "")
```

Parameters

**handleId** [string](#)

装置接続ID / Device connection ID

**mgcParamLogFolder** [string](#)

ゲイン調整ログフォルダ / Gain adjustment log folder

Returns

[Task](#) <[EnmSdkResult](#)>

結果コード / Result code

## RequestHardwareInfoAsync(string, EnmSdkDeviceInfo)

装置からハードウェア情報を取得 / Get hardware info from device

```
public Task<EnmSdkResult> RequestHardwareInfoAsync(string handleId, EnmSdkDeviceInfo type)
```

Parameters

**handleId** [string](#)

装置接続ID / Device connection ID

**type** [EnmSdkDeviceInfo](#)

デバイス情報区分 / Device info type

Returns

[Task](#) <[EnmSdkResult](#)>

結果コード / Result code

## RequestHardwareStateAsync(string, EnmSdkDeviceState)

装置からハードウェア状態を取得 / Get hardware state from device

```
public Task<EnmSdkResult> RequestHardwareStateAsync(string handleId, EnmSdkDeviceState type)
```

Parameters

**handleId** [string](#)

装置接続ID / Device connection ID

**type** [EnmSdkDeviceState](#)

デバイス状態区分 / Device state type

Returns

[Task](#) <[EnmSdkResult](#)>

結果コード / Result code

## ResetBluetoothModuleAsync()

PC内蔵もしくは外付けのBluetooth機能をリセットする / Reset built-in or external Bluetooth function

```
public Task ResetBluetoothModuleAsync()
```

Returns

[Task](#)

## ResetToDefaultParametersAsync(string)

デフォルト設定にリセットする / Reset to default parameters

```
public Task ResetToDefaultParametersAsync(string handleId)
```

Parameters

**handleId** [string](#)

装置接続ID / Device connection ID

Returns

[Task](#)

## ResetToDefaultParametersFullAsync(string, EnmSdkDeviceType, string)

全機能をデフォルト設定へリセットする / Reset all functions to default

```
public Task ResetToDefaultParametersFullAsync(string handleId, EnmSdkDeviceType deviceType,  
string lotId = "")
```

Parameters

**handleId** [string](#)

装置接続ID / Device connection ID

**deviceType** [EnmSdkDeviceType](#)

デバイス機種 / Device type

lotId [string](#)

ロットID / Lot ID

Returns

[Task](#)

## SearchAndConnectDeviceAsync(EnmSdkConnectType, int)

装置の検索と接続 / Device search and connect

```
public Task<EnmSdkResult> SearchAndConnectDeviceAsync(EnmSdkConnectType connectType,  
int timeoutSecond)
```

Parameters

connectType [EnmSdkConnectType](#)

接続方式 / Connection type

timeoutSecond [int](#)

検索タイムアウト（秒） / Search timeout (seconds)

Returns

[Task](#) <[EnmSdkResult](#)>

接続結果 / Connection result

## SetBlockMarker(string, bool)

ブロックマーカを設定する / Set block marker

```
public void SetBlockMarker(string markerText, bool markerActive)
```

Parameters

markerText [string](#)

マーカー文字列 / Marker string

markerActive [bool](#)

マーカー有効/無効 / Marker enabled/disabled

## SetBluetoothEnableAsync(bool)

PC内蔵もしくは外付けのBluetooth機能をON/OFFする / Enable/disable built-in or external Bluetooth function

```
public Task SetBluetoothEnableAsync(bool onOff)
```

### Parameters

onOff [bool](#)

T:ON, F:OFF / T:ON, F:OFF

### Returns

[Task](#)

## SetDeviceNameAsync(string, string)

デバイス機器名称更新 / Update device name

```
public Task<EnmSdkResult> SetDeviceNameAsync(string handleId, string devname)
```

### Parameters

handleId [string](#)

装置接続ID / Device connection ID

devname [string](#)

デバイス名称(最大8文字) / Device name (max 8 chars)

## Returns

[Task](#) <[EnmSdkResult](#)>

結果コード / Result code

## SetEventMarker(string)

イベントマーカを設定する / Set event marker

```
public void SetEventMarker(string markerText)
```

## Parameters

markerText [string](#)

マーカ文字列 / Marker string

## SetIRInitSettingAsync(string, int[], int[], int[], int[])

待機時IR設定の更新 / Update IR settings during standby

```
public Task<EnmSdkResult> SetIRInitSettingAsync(string handleId, int[] swGains, int[]  
pwGains, int[] ioValues, int[] brights)
```

## Parameters

handleId [string](#)

装置接続ID / Device connection ID

swGains [int](#)[]

SWゲイン配列 / SW gain array

pwGains [int](#)[]

PWゲイン配列 / PW gain array

ioValues [int](#)[]

IO値配列 / IO values array

`brights` [int](#)[]

明るさ配列 / Brightness array

Returns

[Task](#) <[EnmSdkResult](#)>

結果コード / Result code

## SetMgcGainAsync(string, int, int, int, int)

ゲイン値更新 / Update gain values

```
public Task<EnmSdkResult> SetMgcGainAsync(string handleId, int gainL1, int gainL3, int gainR1, int gainR3)
```

Parameters

`handleId` [string](#)

装置接続ID / Device connection ID

`gainL1` [int](#)

左L1ゲイン / Left L1 gain

`gainL3` [int](#)

左L3ゲイン / Left L3 gain

`gainR1` [int](#)

右R1ゲイン / Right R1 gain

`gainR3` [int](#)

右R3ゲイン / Right R3 gain

Returns



[Task](#) <[EnmSdkResult](#)>

結果コード / Result code

## SetMgcLedAsync(string, int, int)

赤外線光量値更新 / Update infrared LED brightness

```
public Task<EnmSdkResult> SetMgcLedAsync(string handleId, int ledBrightnessL,  
int ledBrightnessR)
```

### Parameters

handleId [string](#)

装置接続ID / Device connection ID

ledBrightnessL [int](#)

左LED光量 / Left LED brightness

ledBrightnessR [int](#)

右LED光量 / Right LED brightness

### Returns

[Task](#) <[EnmSdkResult](#)>

結果コード / Result code

## SetMotionRangeAccAsync(string, EnmSdkAccRange)

モーション加速度レンジ更新 / Update motion acceleration range

```
public Task<EnmSdkResult> SetMotionRangeAccAsync(string handleId, EnmSdkAccRange range)
```

### Parameters

handleId [string](#)

装置接続ID / Device connection ID

range [EnmSdkAccRange](#)

モーション加速度レンジ区分 / Motion acceleration range type

Returns

[Task](#) <[EnmSdkResult](#)>

結果コード / Result code

## SetMotionRangeGyroAsync(string, EnmSdkGyroRange)

モーション角速度レンジ更新 / Update motion gyro range

```
public Task<EnmSdkResult> SetMotionRangeGyroAsync(string handleId, EnmSdkGyroRange range)
```

Parameters

handleId [string](#)

装置接続ID / Device connection ID

range [EnmSdkGyroRange](#)

モーション角速度レンジ区分 / Motion gyro range type

Returns

[Task](#) <[EnmSdkResult](#)>

結果コード / Result code

## SetUsbRTOutputModeAsync(string, bool)

リアルタイムUSB出力機能更新 / Update real-time USB output mode

```
public Task<EnmSdkResult> SetUsbRTOutputModeAsync(string handleId, bool output)
```

## Parameters

**handleId** [string](#)

装置接続ID / Device connection ID

**output** [bool](#)

T=使用／F=未使用 / T=enabled/F=disabled

## Returns

[Task](#) <[EnmSdkResult](#)>

結果コード / Result code

# StartMeasureAsync(string, string?, NotifyMeasureDatasCallBack, bool, EnmSdkMeasurementCycle, EnmSdkMeasurementMode)

自動計測開始 / Start automatic measurement

```
public Task<EnmSdkResult> StartMeasureAsync(string handleId, string? measureLogFolder,
ExBrainApi.NotifyMeasureDatasCallBack callback, bool recalibrate = false,
EnmSdkMeasurementCycle measurementCycle = EnmSdkMeasurementCycle.e10Hz,
EnmSdkMeasurementMode measurementMode = EnmSdkMeasurementMode.eNormal)
```

## Parameters

**handleId** [string](#)

装置接続ID / Device connection ID

**measureLogFolder** [string](#)

計測結果出力先フォルダパス / Measurement log folder path

**callback** [ExBrainApi.NotifyMeasureDatasCallBack](#)

計測データ通知用コールバック / Callback for measurement data notification

**recalibrate** [bool](#)

再キャリブレーションフラグ / Recalibration flag

**measurementCycle** [EnmSdkMeasurementCycle](#)

計測サイクル / Measurement cycle

**measurementMode** [EnmSdkMeasurementMode](#)

計測モード / Measurement mode

Returns

[Task](#) [Task](#) <[EnmSdkResult](#)>

結果コード / Result code

## StartSearchDevice(ExBrainSearchParameter, NotifySearchDevicesCallBack?)

装置検索開始 / Start device search

```
public void StartSearchDevice(ExBrainSearchParameter param,  
    ExBrainApi.NotifySearchDevicesCallBack? callback = null)
```

Parameters

**param** [ExBrainSearchParameter](#)

検索パラメータ / Search parameter

**callback** [ExBrainApi.NotifySearchDevicesCallBack](#)

検索結果通知用コールバック / Callback for search result notification

## StopMeasureAsync(string)

自動計測停止 / Stop measurement

```
public Task<EnmSdkResult> StopMeasureAsync(string handleId)
```

Parameters

handleId [string](#)

装置接続ID / Device connection ID

Returns

[Task](#) <[EnmSdkResult](#)>

結果コード / Result code

## StopSearchDevice()

装置検索停止 / Stop device search

```
public void StopSearchDevice()
```

# Delegate

## ExBrainApi.NotifyMeasureDatasCallBack

Namespace: [ExBrainSdk](#)

Assembly: ExBrainSdk.dll

装置計測データ通知用コールバック関数 / Callback for device measurement data notification

```
public delegate void ExBrainApi.NotifyMeasureDatasCallBack(string id,  
ExBrainMeasureData measure_data)
```

### Parameters

**id** [string](#) 

装置接続ID / Device connection ID

**measure\_data** [ExBrainMeasureData](#)

計測データ / Measurement data

# Delegate

## ExBrainApi.NotifyProcessMessageCallBack

Namespace: [ExBrainSdk](#)

Assembly: ExBrainSdk.dll

テキストメッセージ通知用コールバック関数 / Callback for text message notification

```
public delegate void ExBrainApi.NotifyProcessMessageCallBack(string id, string message,
    EnmSdkMessageType type)
```

### Parameters

**id** [string](#) 

装置接続ID / Device connection ID

**message** [string](#) 

メッセージ内容 / Message content

**type** [EnmSdkMessageType](#)

メッセージ種別 / Message type

# Delegate ExBrainApi.NotifySearchDevicesCallBack

Namespace: [ExBrainSdk](#)

Assembly: ExBrainSdk.dll

装置検索結果通知用コールバック関数 / Callback for device search result notification

```
public delegate void ExBrainApi.NotifySearchDevicesCallBack(EnmSdkSearchEvent ev,  
DeviceInformationEx device)
```

## Parameters

**ev** [EnmSdkSearchEvent](#)

検索イベント / Search event

**device** [DeviceInformationEx](#)

デバイス情報 / Device information



# Delegate

## ExBrainApi.NotifyStatusChangedCallback

Namespace: [ExBrainSdk](#)

Assembly: ExBrainSdk.dll

状態変化通知用コールバック関数 / Callback for status change notification

```
public delegate void ExBrainApi.NotifyStatusChangedCallback(string id, EnmSdkStatus status)
```

### Parameters

**id** [string](#) 

装置接続ID / Device connection ID

**status** [EnmSdkStatus](#)

状態値 / Status value

# Namespace ExBrainSdk.Datas

## Classes

### [BatteryState](#)

バッテリー状態 / Battery state

### [DeviceHwInfo](#)

ハードウェア装置情報 / Hardware device information

### [DeviceHwState](#)

ハードウェア装置状態 / Device hardware state

### [DeviceInformationEx](#)

拡張装置情報データ / Extended device information

### [ExBrainCalibration](#)

脳計測キャリブレーションデータ / Brain measurement calibration data

### [ExBrainDiscoveredDevice](#)

発見された装置データ / Discovered device information

### [ExBrainGainInfo](#)

ゲイン情報 / Gain information

### [ExBrainMeasureData](#)

装置計測結果データ / Device measurement result data

### [ExBrainSearchParameter](#)

装置検索パラメータ / Device search parameters

# Class BatteryState

Namespace: [ExBrainSdk.Datas](#)

Assembly: ExBrainSdk.dll

バッテリー状態 / Battery state

```
public class BatteryState
```

## Inheritance

[object](#) ← BatteryState

## Inherited Members

[object.Equals\(object\)](#), [object.Equals\(object, object\)](#), [object.GetHashCode\(\)](#), [object.GetType\(\)](#), [object.MemberwiseClone\(\)](#), [object.ReferenceEquals\(object, object\)](#), [object.ToString\(\)](#)

## Properties

### BatteryGauge

電池残量 [%] / Battery gauge [%]

```
public double BatteryGauge { get; }
```

Property Value

[double](#)

### BatteryVolt

電池電圧 [mV] / Battery voltage [mV]

```
public double BatteryVolt { get; }
```

Property Value



# Class DeviceHwInfo

Namespace: [ExBrainSdk.Datas](#)

Assembly: ExBrainSdk.dll

ハードウェア装置情報 / Hardware device information

```
public class DeviceHwInfo
```

## Inheritance

[object](#) ← DeviceHwInfo

## Inherited Members

[object.Equals\(object\)](#), [object.Equals\(object, object\)](#), [object.GetHashCode\(\)](#), [object.GetType\(\)](#), [object.MemberwiseClone\(\)](#), [object.ReferenceEquals\(object, object\)](#), [object.ToString\(\)](#)

# Properties

## DeviceFwVersion

ファームウェアバージョン / Firmware version

```
public string DeviceFwVersion { get; }
```

Property Value

[string](#)

## DeviceId

機器 ID / Device ID

```
public string DeviceId { get; }
```

Property Value

[string](#)

## DeviceName

機器名称 / Device name

```
public string DeviceName { get; }
```

Property Value

[string](#)

## DeviceType

機種区分 / Device type

```
public EnmSdkDeviceType DeviceType { get; }
```

Property Value

[EnmSdkDeviceType](#)

## EnabledOutputUsb

USB リアルタイム出力機能有無 / USB real-time output enabled flag

```
public bool EnabledOutputUsb { get; }
```

Property Value

[bool](#)

## LotId

ロット ID / Lot ID

```
public string LotId { get; }
```

Property Value

[string](#)

## RangeAcc

モーション加速度レンジ / Motion acceleration range

```
public EnmSdkAccRange RangeAcc { get; }
```

Property Value

[EnmSdkAccRange](#)

## RangeGyro

モーション角速度レンジ / Motion gyro range

```
public EnmSdkGyroRange RangeGyro { get; }
```

Property Value

[EnmSdkGyroRange](#)

## ResoluteAcc

モーション加速度分解能  $\times 10^{-3}$  [G/bit] / Motion acceleration resolution  $\times 10^{-3}$  [G/bit]

```
public double ResoluteAcc { get; }
```

Property Value

[double](#)

# ResoluteGyro

モーション角速度分解能 [DPS/bit] / Motion gyro resolution [DPS/bit]

```
public double ResoluteGyro { get; }
```

Property Value

[double](#)

# ResolutePd

PD 計測値分解能 [mV/bit] / PD resolution [mV/bit]

```
public double ResolutePd { get; }
```

Property Value

[double](#)

# SamplingBattery

バッテリーサンプリング / Battery sampling rate

```
public EnmSdkSamplingBattery SamplingBattery { get; }
```

Property Value

[EnmSdkSamplingBattery](#)

# SamplingMotion

モーションサンプリング / Motion sampling rate

```
public EnmSdkSamplingMotion SamplingMotion { get; }
```



Property Value

[EnmSdkSamplingMotion](#)

## TransRateBattery

バッテリーデータ転送周期 / Battery transfer rate

```
public EnmSdkTransRateBattery TransRateBattery { get; }
```

Property Value

[EnmSdkTransRateBattery](#)

## TransRateHR

心拍数データ転送周期 / Heart-rate transfer rate

```
public EnmSdkTransRateHR TransRateHR { get; }
```

Property Value

[EnmSdkTransRateHR](#)

## TransRateMotion

モーションデータ転送周期 / Motion transfer rate

```
public EnmSdkTransRateMotion TransRateMotion { get; }
```

Property Value

[EnmSdkTransRateMotion](#)

## TransRatePD

脳活動計測データ転送周期 / Brain activity transfer rate

```
public EnmSdkTransRatePD TransRatePD { get; }
```

Property Value

[EnmSdkTransRatePD](#)

## VMaxPd

PD 計測値デジット最大値 / Maximum PD digit value

```
public int VMaxPd { get; }
```

Property Value

[int](#)

# Class DeviceHwState

Namespace: [ExBrainSdk.Datas](#)

Assembly: ExBrainSdk.dll

ハードウェア装置状態 / Device hardware state

```
public class DeviceHwState
```

## Inheritance

[object](#)  ← DeviceHwState

## Inherited Members

[object.Equals\(object\)](#) , [object.Equals\(object, object\)](#) , [object.GetHashCode\(\)](#) , [object.GetType\(\)](#) ,  
[object.MemberwiseClone\(\)](#) , [object.ReferenceEquals\(object, object\)](#) , [object.ToString\(\)](#) 

## Properties

### BatteryTemperature

バッテリー温度 [°C] / Battery temperature [°C]

```
public double BatteryTemperature { get; }
```

Property Value

[double](#) 

### CpuTemperature

CPU 温度 [°C] / CPU temperature [°C]

```
public double CpuTemperature { get; }
```

Property Value

[double](#)

## EnabledBtComm

Bluetooth 通信接続状態 / Bluetooth communication connection state

```
public bool EnabledBtComm { get; }
```

Property Value

[bool](#)

## EnabledBtModule

Bluetooth モジュール使用状態 / Bluetooth module enabled state

```
public bool EnabledBtModule { get; }
```

Property Value

[bool](#)

## EnabledGauge

残量ゲージセンサ検出状態 / Battery gauge sensor detection state

```
public bool EnabledGauge { get; }
```

Property Value

[bool](#)

## EnabledMotion

モーションセンサ検出状態 / Motion sensor detection state

```
public bool EnabledMotion { get; }
```

Property Value

[bool](#)

## EnabledPdL1

PD-L1 値検出状態 / PD-L1 detection state

```
public bool EnabledPdL1 { get; }
```

Property Value

[bool](#)

## EnabledPdL3

PD-L3 値検出状態 / PD-L3 detection state

```
public bool EnabledPdL3 { get; }
```

Property Value

[bool](#)

## EnabledPdR1

PD-R1 値検出状態 / PD-R1 detection state

```
public bool EnabledPdR1 { get; }
```

Property Value

[bool](#)

## EnabledPdR3

PD-R3 値検出状態 / PD-R3 detection state

```
public bool EnabledPdR3 { get; }
```

Property Value

[bool](#)

## EnabledSensModule

センサモジュール検出状態 / Sensor module detection state

```
public bool EnabledSensModule { get; }
```

Property Value

[bool](#)

## EnabledUsbComm

USB 通信接続状態 / USB communication connection state

```
public bool EnabledUsbComm { get; }
```

Property Value

[bool](#)

## EnabledUsbVbus

USB VBUS 状態 / USB VBUS state

```
public bool EnabledUsbVbus { get; }
```

Property Value

[bool](#)

## LedBrightnessL

IR 光量値 (L) / IR brightness value (L)

```
public int LedBrightnessL { get; }
```

Property Value

[int](#)

## LedBrightnessR

IR 光量値 (R) / IR brightness value (R)

```
public int LedBrightnessR { get; }
```

Property Value

[int](#)

## LedEnableL

IR IO 出力状態 (L) / IR IO output state (L)

```
public bool LedEnableL { get; }
```

Property Value

[bool](#)

## LedEnableR

IR IO 出力状態 (R) / IR IO output state (R)

```
public bool LedEnableR { get; }
```

Property Value

[bool](#)

## MotionTemperature

モーション温度 [°C] / Motion temperature [°C]

```
public double MotionTemperature { get; }
```

Property Value

[double](#)



# Class DeviceInformationEx

Namespace: [ExBrainSdk.Datas](#)

Assembly: ExBrainSdk.dll

拡張装置情報データ / Extended device information

```
public class DeviceInformationEx
```

## Inheritance

[object](#)  ← DeviceInformationEx

## Inherited Members

[object.Equals\(object\)](#) , [object.Equals\(object, object\)](#) , [object.GetHashCode\(\)](#) , [object.GetType\(\)](#) ,  
[object.MemberwiseClone\(\)](#) , [object.ReferenceEquals\(object, object\)](#) , [object.ToString\(\)](#) 

## Properties

### ConnectType

接続方式 / Connection type

```
public EnmSdkConnectType ConnectType { get; }
```

Property Value

[EnmSdkConnectType](#)

### DeviceId

接続 ID / Device connection ID

```
public string DeviceId { get; }
```

Property Value

[string](#)

## IsPaired

Bluetooth ペアリング済みフラグ / Bluetooth paired flag

```
public bool IsPaired { get; }
```

Property Value

[bool](#)

## TargetName

接続対象の名称 / Target device name

```
public string TargetName { get; }
```

Property Value

[string](#)

## TargetType

接続対象の機種 / Target device type

```
public EnmSdkDeviceType TargetType { get; }
```

Property Value

[EnmSdkDeviceType](#)

## UsbPort

USB の COM ポート情報 / USB COM port information

```
public string UsbPort { get; }
```

Property Value

[string](#) 

## UsbVidPid

USB の ID 情報 / USB VID/PID information

```
public string UsbVidPid { get; }
```

Property Value

[string](#) 

# Class ExBrainCalibration

Namespace: [ExBrainSdk.Datas](#)

Assembly: ExBrainSdk.dll

脳計測キャリブレーションデータ / Brain measurement calibration data

```
public class ExBrainCalibration
```

## Inheritance

[object](#)  ← ExBrainCalibration

## Inherited Members

[object.Equals\(object\)](#) , [object.Equals\(object, object\)](#) , [object.GetHashCode\(\)](#) , [object.GetType\(\)](#) ,  
[object.MemberwiseClone\(\)](#) , [object.ReferenceEquals\(object, object\)](#) , [object.ToString\(\)](#) 

## Properties

### HbDensityOffsetL1

左 1cm のヘモグロビン密度オフセット / Left 1cm hemoglobin density offset

```
public double HbDensityOffsetL1 { get; set; }
```

Property Value

[double](#) 

### HbDensityOffsetL3

左 3cm のヘモグロビン密度オフセット / Left 3cm hemoglobin density offset

```
public double HbDensityOffsetL3 { get; set; }
```

Property Value

[double](#)

## HbDensityOffsetR1

右 1cm のヘモグロビン密度オフセット / Right 1cm hemoglobin density offset

```
public double HbDensityOffsetR1 { get; set; }
```

Property Value

[double](#)

## HbDensityOffsetR3

右 3cm のヘモグロビン密度オフセット / Right 3cm hemoglobin density offset

```
public double HbDensityOffsetR3 { get; set; }
```

Property Value

[double](#)

# Class ExBrainDiscoveredDevice

Namespace: [ExBrainSdk.Datas](#)

Assembly: ExBrainSdk.dll

発見された装置データ / Discovered device information

```
public class ExBrainDiscoveredDevice
```

## Inheritance

[object](#) ← ExBrainDiscoveredDevice

## Inherited Members

[object.Equals\(object\)](#), [object.Equals\(object, object\)](#), [object.GetHashCode\(\)](#), [object.GetType\(\)](#), [object.MemberwiseClone\(\)](#), [object.ReferenceEquals\(object, object\)](#), [object.ToString\(\)](#)

## Properties

### DeviceCount

発見された装置数 / Number of discovered devices

```
public int DeviceCount { get; }
```

## Property Value

[int](#)

### Devices

発見された装置リスト / Discovered device list

```
public IReadOnlyList<DeviceInformationEx> Devices { get; }
```

## Property Value

[ReadOnlyList](#) <[DeviceInfoEx](#)>

# Class ExBrainGainInfo

Namespace: [ExBrainSdk.Datas](#)

Assembly: ExBrainSdk.dll

ゲイン情報 / Gain information

```
public class ExBrainGainInfo
```

## Inheritance

[object](#)  ← ExBrainGainInfo

## Inherited Members

[object.Equals\(object\)](#) , [object.Equals\(object, object\)](#) , [object.GetHashCode\(\)](#) , [object.GetType\(\)](#) ,  
[object.MemberwiseClone\(\)](#) , [object.ReferenceEquals\(object, object\)](#) , [object.ToString\(\)](#) 

## Constructors

### ExBrainGainInfo()

コンストラクタ / Constructor

```
public ExBrainGainInfo()
```

## Properties

### Gains

ゲイン設定値 / Gain values

```
public int[] Gains { get; set; }
```

### Property Value

[int](#)  []



## Remarks

0:L1cm 1-16 1:L3cm 1-64 2:R1cm 1-16 3:R3cm 1-64 / 0:L1cm 1-16 1:L3cm 1-64 2:R1cm 1-16 3:R3cm 1-64

## Methods

### Initialize(EnmSdkDeviceType)

初期化 / Initialize gain values

```
public void Initialize(EnmSdkDeviceType devType)
```

## Parameters

devType [EnmSdkDeviceType](#)

デバイス機種 / Device type

# Class ExBrainMeasureData

Namespace: [ExBrainSdk.Datas](#)

Assembly: ExBrainSdk.dll

装置計測結果データ / Device measurement result data

```
public class ExBrainMeasureData
```

## Inheritance

[object](#) ← ExBrainMeasureData

## Inherited Members

[object.Equals\(object\)](#), [object.Equals\(object, object\)](#), [object.GetHashCode\(\)](#), [object.GetType\(\)](#), [object.MemberwiseClone\(\)](#), [object.ReferenceEquals\(object, object\)](#), [object.ToString\(\)](#)

## Properties

### AccelerationX

X 軸加速度 [G] / X-axis acceleration [G]

```
[JsonInclude]  
public double AccelerationX { get; }
```

Property Value

[double](#)

### AccelerationY

Y 軸加速度 [G] / Y-axis acceleration [G]

```
[JsonInclude]  
public double AccelerationY { get; }
```

Property Value

[double](#)

## AccelerationZ

Z 軸加速度 [G] / Z-axis acceleration [G]

[JsonInclude]

```
public double AccelerationZ { get; }
```

Property Value

[double](#)

## AngularVelocityX

X 軸角速度 [DPS] / X-axis angular velocity [DPS]

[JsonInclude]

```
public double AngularVelocityX { get; }
```

Property Value

[double](#)

## AngularVelocityY

Y 軸角速度 [DPS] / Y-axis angular velocity [DPS]

[JsonInclude]

```
public double AngularVelocityY { get; }
```

Property Value

[double](#)

# AngularVelocityZ

Z 軸角速度 [DPS] / Z-axis angular velocity [DPS]

```
[JsonInclude]  
public double AngularVelocityZ { get; }
```

Property Value

[double](#)<sup>🔗</sup>

# BrainIndexValueL

左側腦活動指標值 / Left brain activity index value

```
[JsonInclude]  
public double BrainIndexValueL { get; }
```

Property Value

[double](#)<sup>🔗</sup>

# BrainIndexValueR

右側腦活動指標值 / Right brain activity index value

```
[JsonInclude]  
public double BrainIndexValueR { get; }
```

Property Value

[double](#)<sup>🔗</sup>

# ColorIndexLeft

腦活動指標值（左） / Brain activity index (left)

```
[JsonInclude]
public int ColorIndexLeft { get; }
```

Property Value

[int](#)

## ColorIndexRight

脳活動指標値（右） / Brain activity index (right)

```
[JsonInclude]
public int ColorIndexRight { get; }
```

Property Value

[int](#)

## DropCount

前データとの通番抜けカウント / Missing sequence count from previous data

```
[JsonInclude]
public int DropCount { get; }
```

Property Value

[int](#)

## ElapsedSeconds

計測経過時間 [sec] / Elapsed measurement time [sec]

```
[JsonInclude]
public double ElapsedSeconds { get; }
```

Property Value

[double](#)

## HbDensityL1

ヘモグロビン密度 L1 / Hemoglobin Density L1

```
[JsonInclude]  
public double HbDensityL1 { get; }
```

Property Value

[double](#)

## HbDensityL3

ヘモグロビン密度 L3 / Hemoglobin Density L3

```
[JsonInclude]  
public double HbDensityL3 { get; }
```

Property Value

[double](#)

## HbDensityR1

ヘモグロビン密度 R1 / Hemoglobin Density R1

```
[JsonInclude]  
public double HbDensityR1 { get; }
```

Property Value

[double](#)

# HbDensityR3

ヘモグロビン密度 R3 / Hemoglobin Density R3

```
[JsonInclude]  
public double HbDensityR3 { get; }
```

Property Value

[double](#) 

# HeartRate

推定心拍数 [bpm] / Estimated heart rate [bpm]

```
[JsonInclude]  
public double HeartRate { get; }
```

Property Value

[double](#) 

# MarkerText

マーカーテキスト / Marker text

```
[JsonInclude]  
public string MarkerText { get; }
```

Property Value

[string](#) 

# SerialCounter

計測通番カウンタ / Measurement sequence counter

```
[JsonInclude]  
public double SerialCounter { get; }
```

Property Value

[double](#)🔗

## SyntheticAcceleration

合成加速度 [G] / Synthetic acceleration [G]

```
[JsonInclude]  
public double SyntheticAcceleration { get; }
```

Property Value

[double](#)🔗

## UpdateDateTime

更新日時（ローカル時刻） / Updated datetime (local time)

```
[JsonInclude]  
public DateTime UpdateDateTime { get; }
```

Property Value

[DateTime](#)🔗



# Class ExBrainSearchParameter

Namespace: [ExBrainSdk.Datas](#)

Assembly: ExBrainSdk.dll

装置検索パラメータ / Device search parameters

```
public class ExBrainSearchParameter
```

## Inheritance

[object](#)  ← ExBrainSearchParameter

## Inherited Members

[object.Equals\(object\)](#) , [object.Equals\(object, object\)](#) , [object.GetHashCode\(\)](#) , [object.GetType\(\)](#) ,  
[object.MemberwiseClone\(\)](#) , [object.ReferenceEquals\(object, object\)](#) , [object.ToString\(\)](#) 

## Properties

### Keywords

検索キーワード（複数可） / Search keywords (multiple allowed)

```
public string[] Keywords { get; set; }
```

### Property Value

[string](#)  []

### ShouldNonPairedBluetooth

周辺 Bluetooth を検索対象にする / Include non-paired Bluetooth devices in search

```
public bool ShouldNonPairedBluetooth { get; set; }
```

### Property Value

[bool](#)

## ShouldPairedBluetooth

ペアリング済み Bluetooth を検索対象にする / Include paired Bluetooth devices in search

```
public bool ShouldPairedBluetooth { get; set; }
```

Property Value

[bool](#)

## ShouldUSB

USB を検索対象にする / Include USB devices in search

```
public bool ShouldUSB { get; set; }
```

Property Value

[bool](#)

## TimeoutSecond

検索タイムアウト [秒] / Search timeout in seconds

```
public int TimeoutSecond { get; set; }
```

Property Value

[int](#)

# Namespace ExBrainSdk.Enumerations

## Enums

### [EnmSdkAccRange](#)

モーション加速度レンジ区分 / Motion acceleration range categories

### [EnmSdkConnectType](#)

接続方式定義 / Connection type definitions

### [EnmSdkDeviceInfo](#)

デバイス情報区分定義 / Device info categories

### [EnmSdkDeviceState](#)

デバイス状態区分定義 / Device state categories

### [EnmSdkDeviceType](#)

機種区分 / Device type categories

### [EnmSdkDiagRecordType](#)

ダイアグレコードタイプ / Diagnostic record types

### [EnmSdkEvent](#)

動作イベント定義 / SDK event definitions

### [EnmSdkGyroRange](#)

モーション角速度レンジ区分 / Motion gyro range categories

### [EnmSdkMeasurementCycle](#)

計測サイクル定義 / Measurement cycle definitions

### [EnmSdkMeasurementMode](#)

計測モード定義 / Measurement mode definitions

### [EnmSdkMessageType](#)

テキストメッセージタイプ / Text message types

### [EnmSdkPdLocations](#)

計測位置区分 / Measurement locations

### [EnmSdkResult](#)

処理結果定義 / Result code definitions

### [EnmSdkSamplingBattery](#)

バッテリーサンプリング区分 / Battery sampling rate categories

### [EnmSdkSamplingMotion](#)

モーションサンプリング区分 / Motion sampling rate categories

### [EnmSdkSearchEvent](#)

機器検索結果区分 / Device discovery event categories

### [EnmSdkStatus](#)

動作状態定義 / SDK status definitions

### [EnmSdkTransRateBattery](#)

バッテリー転送周期区分 / Battery transfer rate categories

### [EnmSdkTransRateHR](#)

心拍数転送周期区分 / Heart-rate transfer rate categories

### [EnmSdkTransRateMotion](#)

モーション転送周期区分 / Motion transfer rate categories

### [EnmSdkTransRatePD](#)

脳活動計測データ転送周期区分 / Brain activity transfer rate categories

# Enum EnmSdkAccRange

Namespace: [ExBrainSdk.Enumerations](#)

Assembly: ExBrainSdk.dll

モーション加速度レンジ区分 / Motion acceleration range categories

```
public enum EnmSdkAccRange
```

## Fields

e16G = 4

16G / 16G

e2G = 1

2G / 2G

e4G = 2

4G / 4G

e8G = 3

8G / 8G

eDefault = 0

デフォルト / Default

# Enum EnmSdkConnectType

Namespace: [ExBrainSdk.Enumerations](#)

Assembly: ExBrainSdk.dll

接続方式定義 / Connection type definitions

```
public enum EnmSdkConnectType
```

## Fields

**eBluetooth = 1**

Bluetooth / Bluetooth

**eNotSelected = 0**

未選択 / Not selected

**eUsb = 2**

USB / USB

# Enum EnmSdkDeviceInfo

Namespace: [ExBrainSdk.Enumerations](#)

Assembly: ExBrainSdk.dll

デバイス情報区分定義 / Device info categories

```
public enum EnmSdkDeviceInfo
```

## Fields

**eAccSettings = 6**

加速度設定情報 / Acceleration settings

**eAll = 0**

全ての情報 / All information

**eBatteryCapacity = 14**

バッテリー容量設定 / Battery capacity settings

**eDevId = 2**

デバイス ID / Device ID

**eDevName = 3**

デバイス名称 / Device name

**eDevType = 1**

機種 / Device type

**eFwVersion = 12**

ファームウェアバージョン情報 / Firmware version information

**eGyroSettings = 7**

角速度設定情報 / Gyro settings

**eHwSettings = 5**

ハードウェア情報 / Hardware settings

`eInitIrSettings = 13`

起動時近赤外線 LED 設定 / Startup infrared LED settings

`eLotId = 4`

ロット ID / Lot ID

`eMeasureAnalysisSettings = 10`

計測解析設定 / Measurement analysis settings

`eMeasureModeSettings = 8`

計測モード設定 / Measurement mode settings

`eMeasureTransferSettings = 9`

計測転送モード設定 / Measurement transfer settings

`eMinimum = 15`

最低限のデータ取得 / Minimum data acquisition

`eUsbRTOutput = 11`

USB リアルタイム出力設定 / USB real-time output settings



# Enum EnmSdkDeviceState

Namespace: [ExBrainSdk.Enumerations](#)

Assembly: ExBrainSdk.dll

デバイス状態区分定義 / Device state categories

```
public enum EnmSdkDeviceState
```

## Fields

**eAll = 0**

全ての状態 / All states

**eBatteryGauge = 1**

バッテリー残量状態 / Battery gauge state

**eBatteryState = 10**

バッテリー検知状態 / Battery detection state

**eBatteryVolt = 2**

バッテリー電圧状態 / Battery voltage state

**eBluetoothState = 6**

Bluetooth 状態 / Bluetooth state

**eIrLedState = 4**

近赤外線 LED 状態 / Infrared LED state

**eMotionState = 9**

モーション検知状態 / Motion detection state

**ePDState = 8**

受光センサ検知状態 / Photodiode detection state

**eSensorModuleState = 7**

センサモジュール状態 / Sensor module state

eTemperatures = 3

温度状態 / Temperature state

eUsbState = 5

USB 状態 / USB state

# Enum EnmSdkDeviceType

Namespace: [ExBrainSdk.Enumerations](#)

Assembly: ExBrainSdk.dll

機種区分 / Device type categories

```
public enum EnmSdkDeviceType
```

## Fields

eHOT2000 = 1

HOT2000 / HOT2000

eUnknown = -1

不明 / Unknown

eXB01 = 2

XB01 / XB01

# Enum EnmSdkDiagRecordType

Namespace: [ExBrainSdk.Enumerations](#)

Assembly: ExBrainSdk.dll

ダイアグレコードタイプ / Diagnostic record types

```
public enum EnmSdkDiagRecordType
```

## Fields

eDiag = 0

ダイアグレコード / Diagnostic record

eTrace = 1

トレースレコード / Trace record

# Enum EnmSdkEvent

Namespace: [ExBrainSdk.Enumerations](#)

Assembly: ExBrainSdk.dll

動作イベント定義 / SDK event definitions

```
public enum EnmSdkEvent
```

## Fields

**eBluetoothResetCompleted = 5**

Bluetooth モジュールリセット完了 / Bluetooth module reset completed

**eConnectCompleted = 1**

接続完了 / Connection completed

**eDisconnectCompleted = 3**

切断完了 / Disconnection completed

**eDisconnectDetected = 4**

切断検知 / Disconnection detected

**eReconnectCompleted = 2**

再接続完了 / Reconnection completed

# Enum EnmSdkGyroRange

Namespace: [ExBrainSdk.Enumerations](#)

Assembly: ExBrainSdk.dll

モーション角速度レンジ区分 / Motion gyro range categories

```
public enum EnmSdkGyroRange
```

## Fields

e1000DPS = 4

1000DPS / 1000DPS

e125DPS = 1

125DPS / 125DPS

e2000DPS = 5

2000DPS / 2000DPS

e250DPS = 2

250DPS / 250DPS

e500DPS = 3

500DPS / 500DPS

eDefault = 0

デフォルト / Default

# Enum EnmSdkMeasurementCycle

Namespace: [ExBrainSdk.Enumerations](#)

Assembly: ExBrainSdk.dll

計測サイクル定義 / Measurement cycle definitions

```
public enum EnmSdkMeasurementCycle
```

## Fields

e100Hz = 2

100Hz

e10Hz = 0

10Hz

e500Hz = 3

500Hz

e50Hz = 1

50Hz

# Enum EnmSdkMeasurementMode

Namespace: [ExBrainSdk.Enumerations](#)

Assembly: ExBrainSdk.dll

計測モード定義 / Measurement mode definitions

```
public enum EnmSdkMeasurementMode
```

## Fields

```
eHighPrecisionLowReliability = 2
```

高精度・低信頼 / High precision, low reliability

```
eLowPrecisionHighReliability = 1
```

低精度・高信頼 / Low precision, high reliability

```
eNormal = 0
```

通常計測 / Normal measurement



# Enum EnmSdkMessageType

Namespace: [ExBrainSdk.Enumerations](#)

Assembly: ExBrainSdk.dll

テキストメッセージタイプ / Text message types

```
public enum EnmSdkMessageType
```

## Fields

`eError = 2`

エラー / Error

`eNormal = 0`

通常 / Normal

`eStatus = 1`

状態 / Status

# Enum EnmSdkPdLocations

Namespace: [ExBrainSdk.Enumerations](#)

Assembly: ExBrainSdk.dll

計測位置区分 / Measurement locations

```
public enum EnmSdkPdLocations
```

## Fields

**L1cm = 0**

L1cm / L1cm

**L3cm = 1**

L3cm / L3cm

**R1cm = 2**

R1cm / R1cm

**R3cm = 3**

R3cm / R3cm

# Enum EnmSdkResult

Namespace: [ExBrainSdk.Enumerations](#)

Assembly: ExBrainSdk.dll

処理結果定義 / Result code definitions

```
public enum EnmSdkResult
```

## Fields

**eCalibratingDataFullErr = 25**

キャリブレーションデータ満杯エラー / Calibration data full error

**eCalibratingStartErr = 23**

キャリブレーション開始エラー / Calibration start error

**eCalibratingStopErr = 24**

キャリブレーション停止エラー / Calibration stop error

**eFailed = 1**

失敗 / Failed

**eFittingDataFullErr = 16**

装着調整データ満杯エラー / Fitting data full error

**eFittingGainOverErr = 17**

装着調整ゲインオーバーエラー / Fitting gain over error

**eFittingLastCheckErr = 19**

装着調整最終チェックエラー / Fitting final check error

**eFittingLastCheckSaturationOverErr = 20**

装着調整最終チェック飽和オーバーエラー / Fitting final check saturation over error

**eFittingLastCheckValueOverErr = 22**

装着調整最終チェック値オーバーエラー / Fitting final check value over error

`eFittingLastCheckValueUnderErr = 21`

装着調整最終チェック値アンダーエラー / Fitting final check value under error

`eFittingRetryOverErr = 18`

装着調整リトライオーバーエラー / Fitting retry over error

`eFittingStartErr = 14`

装着調整開始エラー / Fitting start error

`eFittingStopErr = 15`

装着調整停止エラー / Fitting stop error

`eLicenseError = 28`

ライセンスエラー / License error

`eMeasureStartErr = 26`

計測開始エラー / Measurement start error

`eMismatchDeviceId = 12`

デバイスID不一致 / Device ID mismatch

`eMismatchStatus = 11`

状態不一致 / Status mismatch

`eNotConnected = 4`

未接続 / Not connected

`eNotFoundDevice = 10`

デバイス未検出 / Device not found

`eNotInitiated = 3`

未初期化 / Not initiated

`eNotSupported = 2`

未サポート / Not supported

`eParamWrong = 9`

パラメータ不正 / Invalid parameter

`ePreSameParam = 13`

前回と同じパラメータのため処理なし / No operation because parameters are unchanged from previous call

`eReadFailed = 6`

読込失敗 / Read failed

`eReadTimeout = 5`

読込タイムアウト / Read timeout

`eSuccess = 0`

成功 / Success

`eSystemError = 27`

システムエラー / System error

`eWriteFailed = 8`

書込失敗 / Write failed

`eWriteTimeout = 7`

書込タイムアウト / Write timeout

# Enum EnmSdkSamplingBattery

Namespace: [ExBrainSdk.Enumerations](#)

Assembly: ExBrainSdk.dll

バッテリーサンプリング区分 / Battery sampling rate categories

```
public enum EnmSdkSamplingBattery
```

## Fields

`e0_1HZ = 1`

0.1Hz / 0.1Hz

`e0_2HZ = 2`

0.2Hz / 0.2Hz

`e0_5HZ = 3`

0.5Hz / 0.5Hz

`e1HZ = 4`

1Hz / 1Hz

`eDisable = 0`

無効 / Disabled

# Enum EnmSdkSamplingMotion

Namespace: [ExBrainSdk.Enumerations](#)

Assembly: ExBrainSdk.dll

モーションサンプリング区分 / Motion sampling rate categories

```
public enum EnmSdkSamplingMotion
```

## Fields

e10HZ = 2

10Hz / 10Hz

e1HZ = 1

1Hz / 1Hz

e20HZ = 3

20Hz / 20Hz

e50HZ = 4

50Hz / 50Hz

eDisable = 0

無効 / Disabled

# Enum EnmSdkSearchEvent

Namespace: [ExBrainSdk.Enumerations](#)

Assembly: ExBrainSdk.dll

機器検索結果区分 / Device discovery event categories

```
public enum EnmSdkSearchEvent
```

## Fields

**eAdded = 0**

追加 / Added

**eRemoved = 2**

削除 / Removed

**eStopped = 3**

停止 / Stopped

**eUpdated = 1**

更新 / Updated



# Enum EnmSdkStatus

Namespace: [ExBrainSdk.Enumerations](#)

Assembly: ExBrainSdk.dll

動作状態定義 / SDK status definitions

```
public enum EnmSdkStatus
```

## Fields

**eCalibrating = 8**

装置準備中（キャリブレーション中） / Preparing device (calibrating)

**eConnecting = 4**

装置接続中 / Connecting device

**eDisconnecting = 5**

装置切断中 / Disconnecting device

**eFitting = 7**

装着調整中 / Fitting

**eIdle = 6**

待機中 / Idle

**eInitial = 1**

初期化中 / Initializing

**eLosting = 10**

通信ロスト中 / Communication lost

**eMeasureLosting = 11**

計測時通信ロスト中 / Communication lost while measuring

**eMeasuring = 9**

装置計測中 / Measuring

eOffline = 2

オフライン / Offline

eSearching = 3

装置検索中 / Searching device

# Enum EnmSdkTransRateBattery

Namespace: [ExBrainSdk.Enumerations](#)

Assembly: ExBrainSdk.dll

バッテリー転送周期区分 / Battery transfer rate categories

```
public enum EnmSdkTransRateBattery
```

## Fields

`e0_1HZ = 1`

0.1Hz / 0.1Hz

`e0_2HZ = 2`

0.2Hz / 0.2Hz

`e0_5HZ = 3`

0.5Hz / 0.5Hz

`e1HZ = 4`

1Hz / 1Hz

`eDisable = 0`

無効 / Disabled

# Enum EnmSdkTransRateHR

Namespace: [ExBrainSdk.Enumerations](#)

Assembly: ExBrainSdk.dll

心拍数転送周期区分 / Heart-rate transfer rate categories

```
public enum EnmSdkTransRateHR
```

## Fields

**e10HZ = 4**

10Hz / 10Hz

**e1HZ = 1**

1Hz / 1Hz

**e2HZ = 2**

2Hz / 2Hz

**e5HZ = 3**

5Hz / 5Hz

**eDisable = 0**

無効 / Disabled

# Enum EnmSdkTransRateMotion

Namespace: [ExBrainSdk.Enumerations](#)

Assembly: ExBrainSdk.dll

モーション転送周期区分 / Motion transfer rate categories

```
public enum EnmSdkTransRateMotion
```

## Fields

**e10HZ = 2**

10Hz / 10Hz

**e1HZ = 1**

1Hz / 1Hz

**e20HZ = 3**

20Hz / 20Hz

**e50HZ = 4**

50Hz / 50Hz

**eDisable = 0**

無効 / Disabled

# Enum EnmSdkTransRatePD

Namespace: [ExBrainSdk.Enumerations](#)

Assembly: ExBrainSdk.dll

脳活動計測データ転送周期区分 / Brain activity transfer rate categories

```
public enum EnmSdkTransRatePD
```

## Fields

**e10HZ = 4**

10Hz / 10Hz

**e1HZ = 1**

1Hz / 1Hz

**e2HZ = 2**

2Hz / 2Hz

**e5HZ = 3**

5Hz / 5Hz

**eDisable = 0**

無効 / Disabled